UNIVERSIDAD NACIONAL DEL ALTIPLANO

Escuela Profesional de Ingeniería de Sistemas

INFORME DE PRÁCTICA DE LABORATORIO

SEMANA 5

Manejo de Excepciones y Diseño por Contrato

Jerarquías de errores · Precondiciones, poscondiciones e invariantes · Patrón Result

Estudiante	**Kevin Manuel Vilca Paco**
Docente	Dr. Aldo Hernán Zanabria Gálvez
Curso	Programación Orientada a Objetos II — SIS206
Semestre	2026-II
Lenguajes	C++17 y Python 3.12

Puno — Perú — 2026

Contenido

1. Introducción y objetivos
2. Estructura del proyecto
3. Diagnóstico del código del marco teórico
4. Actividad 1 — Jerarquía de excepciones (C++ y Python)
5. Actividad 2 — Diseño por Contrato: clase Matricula
6. Actividad 3 — Patrón Result y pipeline encadenado
7. Actividad 4 — Verificación de contratos con pruebas unitarias
8. Actividad 5 — Proyecto Integrador: robustez en todas las capas
9. Preguntas de reflexión
10. Cumplimiento de la rúbrica
11. Conclusiones
12. Referencias

1. Introducción y objetivos

Un programa robusto no es el que nunca falla, sino el que **falla de forma controlada**: avisa qué pasó, dónde y con qué datos, y deja los objetos en un estado válido. En esta práctica se construyen módulos resistentes a condiciones anómalas en **C++17** y **Python 3.12**, combinando tres herramientas:

- **Jerarquías de excepciones** propias del dominio académico, que transportan contexto (valor inválido, código, límite superado).
- **Diseño por Contrato (DbC)**: precondiciones (lo que debe cumplir quien llama), poscondiciones (lo que promete el método) e invariantes (lo que el objeto cumple siempre).
- **Patrón Result**: para los fallos esperados (una búsqueda sin resultado, una regla de negocio), el error viaja como valor y no como excepción.

Antes de desarrollar las actividades se **compiló y ejecutó el código del marco teórico** para diagnosticarlo (sección 3); luego se implementaron las cinco actividades y se capturó la salida real de cada programa.

Cuándo usar cada mecanismo

Tipo de fallo	Ejemplo en esta práctica	Mecanismo
Bug del programador (contrato roto por dentro)	Invariante de Curso rota	`assert`
Error de uso o de entorno	Matricular con 7 cursos; archivo inexistente	`throw` / `raise`
Fallo de negocio esperado	Estudiante no encontrado; PPA insuficiente	`Result<T>`

2. Estructura del proyecto

Todo el trabajo está en una sola carpeta (`semana5_kevin_vilca`), organizada por lenguaje y por actividad:

ÁRBOL DE CARPETAS

```
semana5_kevin_vilca/
├── README.md · DIAGNOSTICO.md · Makefile · .gitignore
├── cpp/
│   ├── include/        errores_academicos.hpp · result.hpp · curso.hpp · matricula.hpp
│   ├── marco_teorico/  demo_excepciones.cpp · demo_curso.cpp · demo_result.cpp
│   ├── actividad1/     main.cpp
│   ├── actividad2/     main.cpp
│   ├── actividad3/     main.cpp
│   ├── actividad4/     tests_matricula.cpp
│   └── actividad5/     errores_dominio.hpp · modelos.hpp · main.cpp · tests_dominio.cpp
├── python/
│   ├── excepciones.py · result.py · curso.py · matricula.py
│   ├── marco_01_excepciones.py · marco_02_curso.py · marco_03_result.py
│   ├── actividad1_jerarquia.py · actividad2_matricula.py · actividad3_pipeline.py
│   ├── proyecto_integrador/   excepciones.py · modelos.py · demo.py
│   └── tests/                 test_matricula.py · test_dominio.py
└── informe/            este informe (.docx y .pdf) · uml/ · salidas/
```

Herramientas utilizadas

Herramienta	Uso
g++ 13 (C++17)	`g++ -std=c++17 -Wall -Wextra` — compilación sin advertencias; también con `-fsanitize=address,undefined`
Python 3.12	Implementación Python y comparación idiomática
pytest	`pytest tests -v` — pruebas de contratos en Python
Graphviz	Diagramas UML de este informe

3. Diagnóstico del código del marco teórico

El código del marco teórico se copió tal cual y se intentó compilar y ejecutar. Primero se muestra cómo se comporta el código **ya corregido** (los seis programas de demostración) y después la lista de problemas encontrados.

3.1 Programas del marco teórico ejecutados

C++ — captura selectiva (marco 2.2)

▶ SALIDA DEL PROGRAMA

```
--- Nota 25 (fuera de rango) ---
[catch ErrorCalificacion] valor=25
  [Académico] [Validación] Calificación 25.00 fuera de rango [0, 20]
--- Nota 15 (válida) ---
Sin errores
--- Capacidad agotada ---
[catch ErrorAcademico] [Académico] SIS206: capacidad agotada (30/30)
--- Recurso inexistente ---
[catch ErrorAcademico] [Académico] Estudiante id=9999 no encontrado
--- Error estándar ---
[catch std::exception] índice fuera de rango
```

Análisis. La nota 25 la atrapa el `catch` más específico (`ErrorCalificacion`), por eso se puede leer `e.valor()`. La capacidad agotada y el recurso inexistente no tienen `catch` propio y caen en `ErrorAcademico` (la raíz). El `out_of_range` de la biblioteca estándar solo lo atrapa el último. El orden importa: gana el primer `catch` que coincide.

C++ — Diseño por Contrato con Curso (marco 2.4)

▶ SALIDA DEL PROGRAMA

```
Curso SIS206 creado: cupos libres = 2
Inscritos: 2/2
Precondición: [Académico] SIS206: capacidad agotada (2/2)  (2/2)
Constructor: [Académico] [Validación] Créditos deben estar en [1, 6]
Constructor: [Académico] [Validación] Código del curso no puede ser vacío
```

Análisis. El tercer `inscribir()` viola la precondición (cupos agotados) y lanza `ErrorCapacidad` con los datos 2/2. El constructor rechaza créditos 9 y código vacío **antes** de crear el objeto, así que nunca existe un `Curso` inválido.

C++ — patrón Result (marco 2.5)

▶ SALIDA DEL PROGRAMA

```
Encontrado: Juan Mamani
Error: Código 9999 no encontrado
Longitud del nombre (map): 11
map sobre fallo se propaga: Código 9999 no encontrado
valor() sobre fallo: Result fallido: Código 9999 no encontrado
```

Análisis. La búsqueda exitosa y la fallida se manejan sin `try/catch`. `map()` transforma el valor (11 letras) solo cuando hubo éxito; con un fallo, el error se propaga intacto. Pedir `valor()` de un fallo es un error de uso y lanza `logic_error`.

Python — jerarquía y encadenamiento (marco 2.3)

▶️ SALIDA DEL PROGRAMA

```
--- Nota 25 (fuera de rango) ---
[except ErrorCalificacion] Calificación 25.0 fuera de [0, 20]  <- contexto: {campo='POO II',
valor=25.0, rango='[0, 20]'}
--- Nota 15 (válida) ---
Sin errores
--- Capacidad agotada ---
[except ErrorAcademico] SIS206: capacidad agotada (30/30)  <- contexto: {actual=30, maximo=30,
entidad='SIS206'}

--- Encadenamiento: raise ... from ... ---
Error de dominio : Nota con id=1001 no encontrado  <- contexto: {tipo='Nota', id=1001}
Causa original   : FileNotFoundError: [Errno 2] No such file or directory:
'/ruta/que/no/existe.txt'
Error de dominio : Nota con id=1001 no encontrado  <- contexto: {tipo='Nota', id=1001}
Causa original   : ValueError: could not convert string to float: 'abc'
```

Análisis. El método `__str__` agrega el contexto (campo, valor, rango) al mensaje. Con `raise ... from ...` la excepción de dominio conserva la **causa original** (`FileNotFoundError` o `ValueError`) en `__cause__`, lo que facilita el diagnóstico.

Python — Diseño por Contrato con Curso (marco 2.4)

▶️ SALIDA DEL PROGRAMA

```
Curso SIS206 creado: cupos libres = 2
Inscritos: 2
Diagnóstico: SIS206: OK — 2/2
Precondición: SIS206: capacidad agotada (2/2)  <- contexto: {actual=2, maximo=2,
entidad='SIS206'}
Constructor('SIS999', 9, 10): Créditos 9 fuera de [1, 6]
Constructor('', 3, 10): Código del curso no puede ser vacío
Constructor('SIS500', 3, 0): cupos_max debe ser >= 1
Tras corromper el estado: SIS206: INVÁLIDO — INV: inscritos
```

Análisis. `diagnostico()` informa `OK` con el estado sano. Al forzar `_inscritos = 99` (simulando un bug interno) detecta la invariante rota y lo informa sin lanzar excepción, algo muy útil para depurar.

Python — patrón Result (marco 2.5)

▶️ SALIDA DEL PROGRAMA

```
codigo=1001 -> Beca asignada: Juan Mamani
codigo=1002 -> No se pudo asignar beca: PPA 12.0 < 14.0 (mínimo para beca)
codigo=9999 -> No se pudo asignar beca: Código 9999 no existe

map() transforma un valor; and_then() encadena pasos que devuelven Result:
  map      -> JUAN MAMANI
  map fallo-> Código 9999 no existe
  valor sobre fallo -> Result fallido: Código 9999 no existe
```

Análisis. El código 1001 recibe la beca (PPA 15). El 1002 falla en el segundo paso (PPA 12 < 14) y el 9999 falla en el primero; en ambos casos los pasos siguientes **no se ejecutan** y el error llega al final sin ningún `try/except`.

3.2 Problemas encontrados en el código original

#	Ubicación	Problema detectado	Corrección aplicada
1	`result.hpp` (2.5)	**No compila con ¦Result⟨std::string⟩¦**: los constructores `Result(T)` y `Result(std::string)` tienen la misma firma; además `holds_alternative⟨T⟩` es ambiguo en `variant⟨string, string⟩`.	Se accede al `variant` por **índice** (`in_place_index`, `index()`, `std::get⟨0/1⟩`).
2	Uso de `result.hpp`	Faltan `#include ⟨unordered_map⟩` y `⟨iostream⟩` (`cout` y `cerr` no declarados).	Includes agregados.
3	`Matricula::diagnostico()`	`"texto" + (cond ? "A" : "B")` suma dos `const char*`: error de compilación.	El estado se guarda antes en un `std::string`.
4	`Matricula` C++ (Act. 2)	`agregarCurso` y la verificación de duplicados están como `TODO` (la prueba de duplicados fallaría). La invariante exige fecha no futura y la clase C++ no tenía fecha.	Se implementaron `agregarCurso`, `eliminarCurso`, duplicados y `fecha_`.
5	`Result::map`	Semántica distinta: en C++ `map` **envuelve** el valor; en Python `map` **encadena**. Un `map` C++ con función que devuelve `Result` produce `Result⟨Result⟨T⟩⟩`.	`map` (transforma) y `andThen` / `and_then` (encadena) en ambos lenguajes.
6	f-strings de `result.py`	`f"PPA {est["ppa"]}"` repite comillas: solo válido en Python ≥ 3.12 (en Colab 3.10/3.11 da `SyntaxError`).	Comillas simples dentro del f-string.
7	Nombres de archivo	El texto usa `errores_academicos.py`, pero `Matricula` importa `excepciones`; la Act. 1 menciona `excepciones_dominio.hpp`.	Un nombre por lenguaje: `errores_academicos.hpp` y `excepciones.py`.
8	`std::to_string(float)`	Imprime `25.000000` en los mensajes de error.	Formato con `ostringstream` → `25.00`.
9	Includes de `errores_academicos.hpp`	Usa `std::move` sin `⟨utility⟩`.	Include agregado.
10	`Result` (C++)	Un `Result` ignorado pierde el error en silencio.	`class [[nodiscard]] Result`: el compilador avisa si se descarta.

> **Resultado tras las correcciones.** Todo el código C++ compila **sin advertencias** con `-std=c++17 -Wall -Wextra -Wpedantic` (también con `-DNDEBUG`) y las pruebas pasan con AddressSanitizer y UBSan, sin fugas ni comportamiento indefinido.

3.3 Evidencia de los errores de compilación

▶️ SALIDA DEL COMPILADOR (CÓDIGO ORIGINAL)

```
$ g++ -std=c++17 -Wall -Wextra result.hpp   (código original del marco 2.5, con T =
std::string)
result.hpp:26:14: error: 'Result<T>::Result(std::string)' cannot be overloaded with
'Result<T>::Result(T)'
                        [with T = std::string]
result.hpp:37:29: error: 'cout' is not a member of 'std'
result.hpp:38:15: error: 'cerr' is not a member of 'std'
```

```
$ g++ -std=c++17 -Wall -Wextra matricula.hpp  (código original de la Actividad 2)
matricula.hpp:15:30: error: invalid operands of types 'const char [13]' and 'const char*'
                          to binary 'operator+'
```

4. Actividad 1 — Jerarquía de excepciones (C++ y Python)

Objetivo. Implementar la jerarquía `ErrorAcademico → ErrorValidacion → ErrorCalificacion` en C++17, agregar `ErrorMatriculaDuplicada` (hija de `ErrorValidacion`), crear `validar_matricula` y demostrar la captura selectiva; después, el equivalente en Python con contexto y encadenamiento.

Archivo	Contenido
`cpp/include/errores_academicos.hpp`	Jerarquía C++ completa (7 clases) con destructor virtual y getters `noexcept`
`cpp/actividad1/main.cpp`	`validar_matricula` y demostración de captura selectiva
`python/excepciones.py`	Jerarquía Python con `contexto` (dict) y `__str__`
`python/actividad1_jerarquia.py`	Demostración con `raise ... from ...`

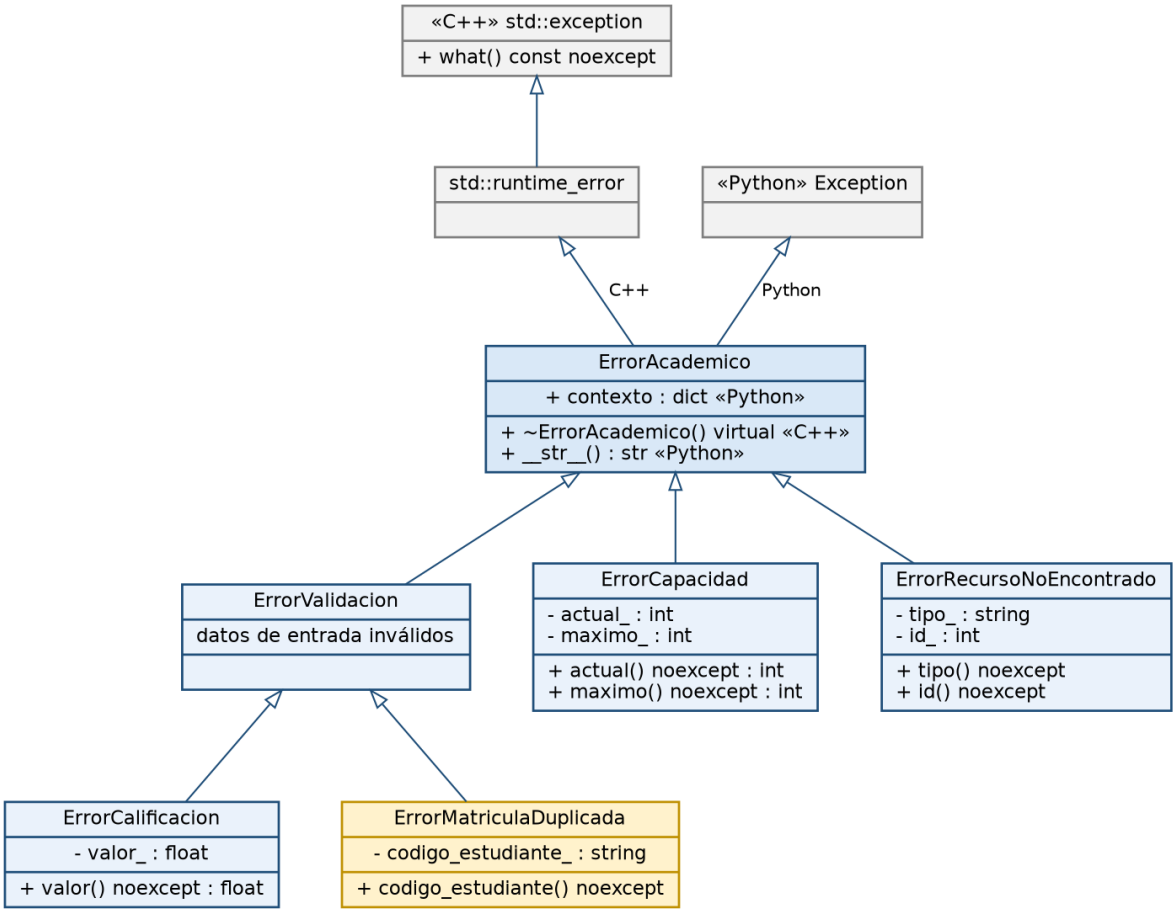


Figura 1. Diagrama UML de la jerarquía de excepciones (C++ y Python). En amarillo, la clase nueva de la Actividad 1.

Parte A — C++17

C++ — cpp/include/errores_academicos.hpp

```
class ErrorMatriculaDuplicada : public ErrorValidacion {
    std::string codigo_estudiante_;
public:
    explicit ErrorMatriculaDuplicada(std::string codigo)
        : ErrorValidacion("Matrícula duplicada para el estudiante " + codigo),
```

```
          codigo_estudiante_(std::move(codigo)) {}
    const std::string& codigo_estudiante() const noexcept { return codigo_estudiante_; }
};
```

C++ — cpp/actividad1/main.cpp

```
void validar_matricula(int codigo, std::vector<int>& matriculados) {
    if (std::find(matriculados.begin(), matriculados.end(), codigo) != matriculados.end())
        throw ErrorMatriculaDuplicada(std::to_string(codigo));
    matriculados.push_back(codigo);
}
```

▶️ **SALIDA — C++**

```
=== Parte A: ErrorMatriculaDuplicada ===
Matriculando código 20210502 ... OK
Matriculando código 20210500 ... DUPLICADA
  catch ErrorMatriculaDuplicada -> codigo_estudiante=20210500
  [Académico] [Validación] Matrícula duplicada para el estudiante 20210500

=== Captura selectiva: una sola rama atrapa a sus hijos ===
ErrorValidacion atrapó una ErrorCalificacion: [Académico] [Validación] Calificación -3.50 fuera
de rango [0, 20]

=== Polimorfismo: captura por referencia a la base ===
what() via ErrorAcademico&: [Académico] matricula: capacidad agotada (6/6)

=== noexcept en los getters ===
noexcept(ec.valor()) = true
```

Parte B — Python 3.12

PYTHON — python/excepciones.py

```
class ErrorMatriculaDuplicada(ErrorValidacion):
    """Actividad 1 — el estudiante ya está matriculado."""

    def __init__(self, codigo_estudiante: str):
        super().__init__(
            f"Matrícula duplicada para el estudiante {codigo_estudiante}",
            contexto={"codigo_estudiante": codigo_estudiante},
        )
        self.codigo_estudiante = codigo_estudiante
```

PYTHON — python/actividad1_jerarquia.py

```
def validar_matricula(codigo: int, matriculados: list[int]) -> None:
    """Lanza ErrorMatriculaDuplicada si el código ya está; si no, lo registra."""
    if codigo in matriculados:
        raise ErrorMatriculaDuplicada(str(codigo))
    matriculados.append(codigo)
```

▶️ **SALIDA — PYTHON**

```
=== Parte B: ErrorMatriculaDuplicada ===
Matriculando código 20210502 ... OK
Matriculando código 20210500 ... DUPLICADA
  except ErrorMatriculaDuplicada -> codigo_estudiante=20210500
  Matrícula duplicada para el estudiante 20210500  <- contexto: {codigo_estudiante='20210500'}

=== Captura por familia ===
ErrorValidacion atrapó una ErrorCalificacion: Calificación -3.5 fuera de [0, 20]  <- contexto:
{campo='POO II', valor=-3.5, rango='[0, 20]'}

=== Contexto rico del error (diccionario) ===
contexto = {'actual': 6, 'maximo': 6, 'entidad': 'matrícula'}

=== raise ... from ...: preserva la causa original ===
```

```
Error de dominio: Nota con id=9999 no encontrado  <- contexto: {tipo='Nota', id=9999}
  __cause__       : KeyError(9999)
```

Tabla comparativa C++17 vs Python 3.12

Aspecto	C++17	Python 3.12
Herencia de clase base	`class ErrorAcademico : public std::runtime_error` (herencia pública de `std::exception`)	`class ErrorAcademico(Exception)`
Acceso a datos del error	Atributos privados + getters tipados `const noexcept` (`valor()`, `codigo_estudiante()`)	Atributos libres y diccionario `e.contexto` accesibles directamente
Garantía de no-lanzar	`noexcept` en los getters; se comprueba con `noexcept(expr)` (salida: `true`)	No existe; solo se documenta (docstring, anotaciones)
Encadenamiento de causa	Manual (`std::nested_exception` / `throw_with_nested`)	`raise X from Y` guarda la causa en `__cause__`
Captura por familia	`catch (const ErrorValidacion&)` atrapa a sus hijas; de específico a general	`except ErrorValidacion as e` atrapa a sus hijas; de específico a general
¿Requiere destructor virtual?	Sí, para el polimorfismo; ya lo hereda de `std::exception` y se declara explícito en la raíz	No aplica: la memoria la gestiona el intérprete

Análisis. `ErrorMatriculaDuplicada` es una `ErrorValidacion`, por eso un `catch ErrorValidacion` también la atraparía; al poner primero el `catch` específico se puede leer `codigo_estudiante`. Python logra lo mismo con menos código (atributos libres y `__str__` con contexto), mientras que C++ ofrece más garantías en tiempo de compilación (`noexcept`, tipos exactos). En ambos lenguajes el orden de captura debe ir de hijo a padre; si el padre va primero, el hijo nunca se ejecuta.

5. Actividad 2 — Diseño por Contrato: clase Matricula

Objetivo. Modelar la matrícula de un estudiante (de 1 a 6 cursos, sin duplicados, fecha no futura, sin modificaciones si está anulada) aplicando precondiciones, poscondiciones e invariante de clase.

Archivo	Contenido
`cpp/include/matricula.hpp`	Clase `Matricula` C++ con DbC completo
`cpp/actividad2/main.cpp`	Demostración de cada caso del contrato
`python/matricula.py`	Clase `Matricula` Python con `_verificar_invariante()`
`python/actividad2_matricula.py`	Demostración equivalente

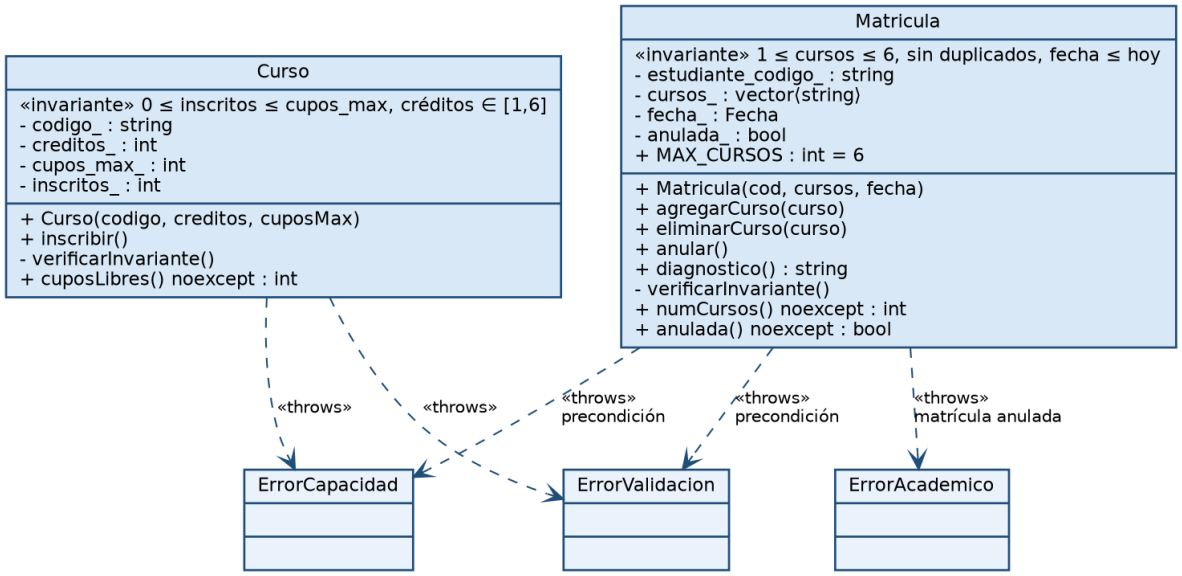


Figura 2. Diagrama UML de Curso y Matricula, con sus dependencias hacia las excepciones.

El contrato de Matricula

Operación	Precondición (throw / raise)	Poscondición (assert)	Invariante
Constructor	Código no vacío · 1 a 6 cursos · sin duplicados · fecha no futura	Objeto en estado válido	Se verifica al final
`agregarCurso`	No anulada · curso no vacío · menos de 6 cursos · no duplicado	Curso presente · tamaño + 1	Se verifica al final
`eliminarCurso`	No anulada · el curso existe · no quedaría vacía	Curso ausente · tamaño − 1	Se verifica al final
`anular`	No estaba anulada	Queda anulada	No se rompe

C++ — cpp/include/matricula.hpp

```
void agregarCurso(const std::string& curso) {
    // ── PRECONDICIONES ──────────────────────────────────────────
    if (anulada_) throw ErrorAcademico("Matrícula anulada — no se puede modificar");
    if (curso.empty()) throw ErrorValidacion("Código de curso vacío");
    if (static_cast<int>(cursos_.size()) >= MAX_CURSOS)
        throw ErrorCapacidad(static_cast<int>(cursos_.size()), MAX_CURSOS, "matricula");
```

```
    if (contiene(curso)) throw ErrorValidacion("Curso '" + curso + "' ya está en la
matrícula");

    [[maybe_unused]] const auto antes = cursos_.size();
    cursos_.push_back(curso);

    // ── POSCONDICIONES ──────────────────────────────────────────
    assert(contiene(curso) && "POST: curso no agregado");
    assert(cursos_.size() == antes + 1 && "POST: el tamaño debe crecer en 1");
    verificarInvariante();
}
```

PYTHON — python/matricula.py

```
def agregar_curso(self, curso: str) -> None:
    # ── PRECONDICIONES ──────────────────────────────────────────
    if self._anulada:
        raise ErrorAcademico("Matrícula anulada — no se puede modificar")
    if not curso.strip():
        raise ErrorValidacion("Código de curso vacío")
    if len(self._cursos) >= self.MAX_CURSOS:
        raise ErrorCapacidad(len(self._cursos), self.MAX_CURSOS, "matrícula")
    if curso in self._cursos:
        raise ErrorValidacion(f"Curso {curso!r} ya está en la matrícula")

    antes = len(self._cursos)
    self._cursos.append(curso)

    # ── POSCONDICIONES ──────────────────────────────────────────
    assert curso in self._cursos, "POST: curso no agregado"
    assert len(self._cursos) == antes + 1, "POST: el tamaño debe crecer en 1"
    assert len(self._cursos) <= self.MAX_CURSOS, "POST: excede máximo"
    self._verificar_invariante()
```

▶️ **SALIDA — C++**

```
=== Constructor (precondiciones) ===
- matrícula válida: OK
- código vacío: ErrorValidacion -> [Académico] [Validación] Código de estudiante vacío
- sin cursos: ErrorCapacidad -> [Académico] matricula: capacidad agotada (0/6)
- 7 cursos: ErrorCapacidad -> [Académico] matricula: capacidad agotada (7/6)
- cursos duplicados: ErrorValidacion -> [Académico] [Validación] Cursos duplicados en la
matrícula
- fecha futura: ErrorValidacion -> [Académico] [Validación] Fecha de matrícula futura no
permitida

=== Mutadores (pre + pos + invariante) ===
Matrícula [ACTIVA] — 1/6 cursos
- agregar SIS207: OK
- agregar SIS207 otra vez: ErrorValidacion -> [Académico] [Validación] Curso 'SIS207' ya está
en la matrícula
- eliminar SIS207: OK
- eliminar el último curso: ErrorValidacion -> [Académico] [Validación] La matrícula no puede
quedar sin cursos (mínimo 1)
Matrícula [ACTIVA] — 6/6 cursos
- agregar un 7.° curso: ErrorCapacidad -> [Académico] matricula: capacidad agotada (6/6)

=== Anulación ===
Matrícula [ANULADA] — 6/6 cursos
- agregar con matrícula anulada: ErrorAcademico -> [Académico] Matrícula anulada — no se puede
modificar
- anular por segunda vez: ErrorAcademico -> [Académico] Matrícula ya está anulada
```

▶️ **SALIDA — PYTHON**

```
=== Constructor (precondiciones) ===
```

```
- matrícula válida: OK
- código vacío: ErrorValidacion -> Código de estudiante vacío
- sin cursos: ErrorCapacidad -> matrícula: capacidad agotada (0/6)  <- contexto: {actual=0,
maximo=6, entidad='matrícula'}
- 7 cursos: ErrorCapacidad -> matrícula: capacidad agotada (7/6)  <- contexto: {actual=7,
maximo=6, entidad='matrícula'}
- cursos duplicados: ErrorValidacion -> Cursos duplicados en la matrícula
- fecha futura: ErrorValidacion -> Fecha futura no permitida: 2026-10-21

=== Mutadores (pre + pos + invariante) ===
Matrícula [ACTIVA] — 1/6 cursos
- agregar SIS207: OK
- agregar SIS207 otra vez: ErrorValidacion -> Curso 'SIS207' ya está en la matrícula
- eliminar SIS207: OK
- eliminar el último curso: ErrorValidacion -> La matrícula no puede quedar sin cursos (mínimo
1)
Matrícula [ACTIVA] — 6/6 cursos
- agregar un 7.° curso: ErrorCapacidad -> matrícula: capacidad agotada (6/6)  <- contexto:
{actual=6, maximo=6, entidad='matrícula'}

=== Anulación ===
Matrícula [ANULADA] — 6/6 cursos
- agregar con matrícula anulada: ErrorAcademico -> Matrícula anulada — no se puede modificar
- anular por segunda vez: ErrorAcademico -> Matrícula ya está anulada

=== diagnostico() detecta una invariante rota ===
INVÁLIDA: INV: duplicados
```

Análisis. Las **precondiciones lanzan excepción** porque dependen de lo que envía el usuario y pueden fallar en producción. Las **poscondiciones e invariantes usan ¦assert¦** porque, si fallan, el culpable es nuestro propio código. Ambos lenguajes dan los mismos resultados. Al final de la salida Python, un duplicado colado saltándose la API (`m2._cursos.append`) es detectado por `diagnostico()` como `INV: duplicados`.

6. Actividad 3 — Patrón Result y pipeline encadenado

Objetivo. Implementar `Result⟨T⟩` con `std::variant` (C++) y con `dataclass + Generic[T]` (Python), y construir el pipeline `buscar_estudiante → validar_ppa_minimo → inscribir` comprobando el caso exitoso (`1001`) y el fallido (`9999`).

Archivo	Contenido
`cpp/include/result.hpp`	`Result⟨T⟩` con `ok`, `fallo`, `esExito`, `valor`, `error`, `map`, `andThen`
`cpp/actividad3/main.cpp`	Pipeline de 3 pasos (C++)
`python/result.py`	`Result` inmutable con los mismos métodos (`and_then` en Python)
`python/actividad3_pipeline.py`	Pipeline de 3 pasos (Python)

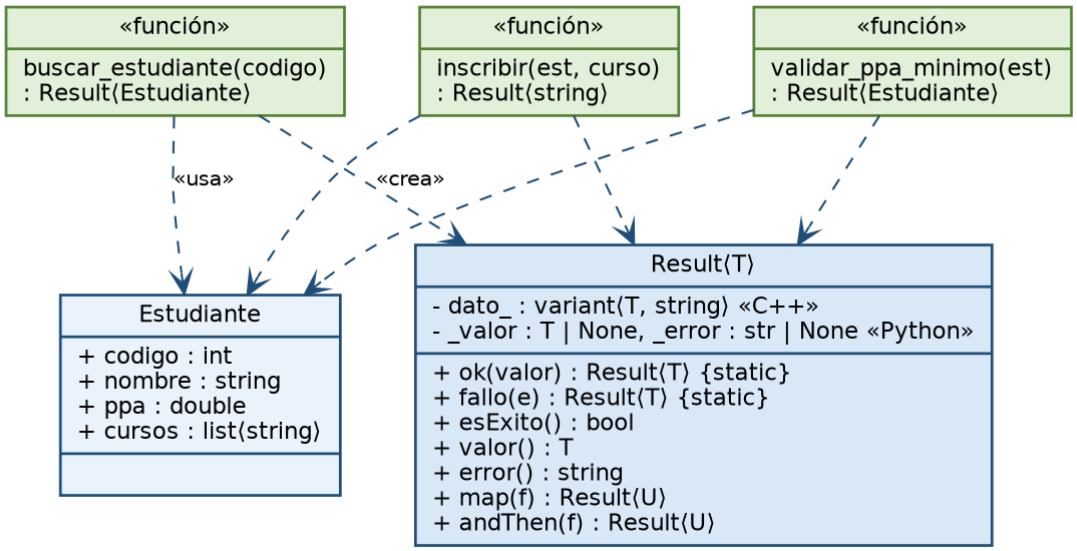


Figura 3. Diagrama UML de Result⟨T⟩ y las funciones del pipeline.

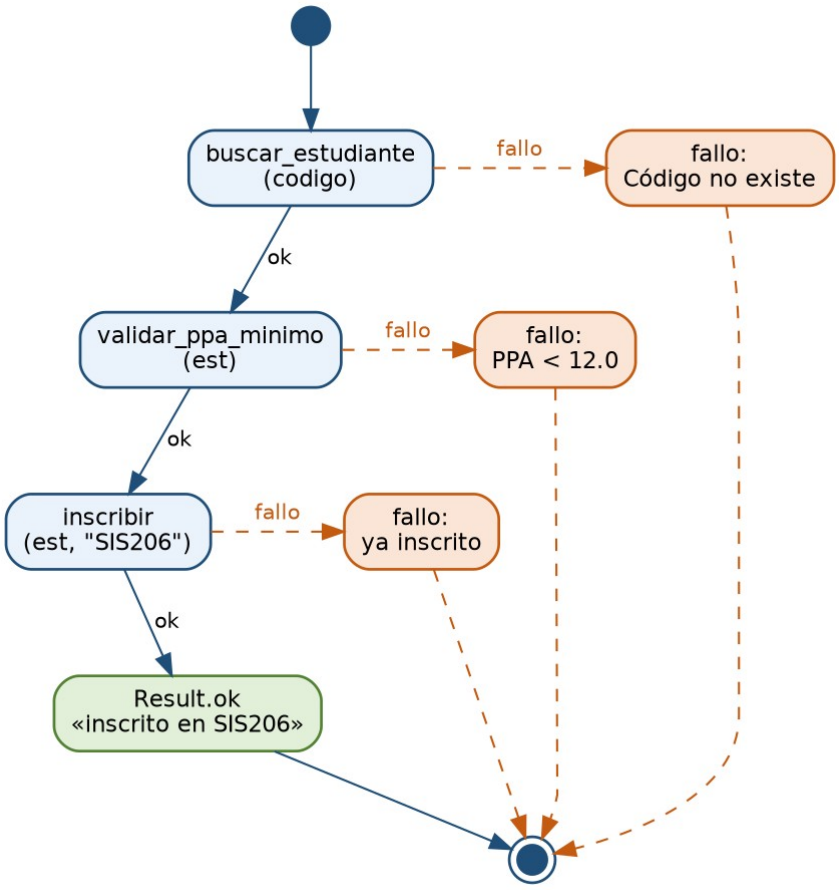


Figura 4. Flujo del pipeline: cada paso devuelve ok o fallo, y el primer fallo cortocircuita el resto.

C++ — cpp/include/result.hpp (estructura)

```
template <typename T>
class [[nodiscard]] Result {
    std::variant<T, std::string> dato_;

    Result(std::in_place_index_t<0> i, T v) : dato_(i, std::move(v)) {}
    Result(std::in_place_index_t<1> i, std::string e) : dato_(i, std::move(e)) {}

public:
    // ── Fábricas ──────────────────────────────────────────────────────
    static Result ok(T valor) { return Result(std::in_place_index<0>, std::move(valor)); }
    static Result fallo(std::string e) { return Result(std::in_place_index<1>, std::move(e)); }

    // ── Consulta ──────────────────────────────────────────────────────
    bool esExito() const noexcept { return dato_.index() == 0; }
```

C++ — cpp/include/result.hpp (map y andThen)

```
    // ── map: transforma el valor (f devuelve un valor normal U) ──────
    template <typename F>
    auto map(F f) const -> Result<std::invoke_result_t<F, const T&>> {
        using U = std::invoke_result_t<F, const T&>;
        if (esExito()) return Result<U>::ok(f(std::get<0>(dato_)));
        return Result<U>::fallo(std::get<1>(dato_));
    }

    // ── andThen: encadena pasos que ya devuelven Result (flatMap) ─────
    template <typename F>
    auto andThen(F f) const -> std::invoke_result_t<F, const T&> {
        using R = std::invoke_result_t<F, const T&>;
```

```
        if (esExito()) return f(std::get<0>(dato_));
        return R::fallo(std::get<1>(dato_));
    }
};
```

PYTHON — python/result.py

```
@dataclass(frozen=True)
class Result(Generic[T]):
    _valor: T | None = None
    _error: str | None = None

    # ── Fábricas ──────────────────────────────────────────────────────
    @staticmethod
    def ok(valor: T) -> "Result[T]":
        return Result(_valor=valor)

    @staticmethod
    def fallo(e: str) -> "Result":
        return Result(_error=e)

    # ── Consulta ──────────────────────────────────────────────────────
    @property
    def es_exito(self) -> bool:
        return self._error is None
```

PYTHON — python/result.py (map y and_then)

```
# ── map: transforma el valor (f devuelve un valor normal) ─────────────
def map(self, f: Callable[[T], U]) -> "Result[U]":
    return Result.ok(f(self._valor)) if self.es_exito else Result.fallo(self._error)  # type:
ignore[arg-type]

# ── and_then: encadena pasos que ya devuelven Result (flatMap) ────────
def and_then(self, f: Callable[[T], "Result[U]"]) -> "Result[U]":
    return f(self._valor) if self.es_exito else Result.fallo(self._error)  # type: ignore[arg-
type]
```

C++ — el pipeline (cpp/actividad3/main.cpp)

```
auto resultado = buscar_estudiante(codigo)
                     .andThen(validar_ppa_minimo)
                     .andThen([](const Estudiante& e) { return inscribir(e, "SIS206"); });
```

PYTHON — el pipeline (python/actividad3_pipeline.py)

```
resultado = (buscar_estudiante(codigo)
             .and_then(validar_ppa_minimo)
             .and_then(lambda e: inscribir(e, "SIS206")))
```

▶ SALIDA — C++

```
Pipeline: buscar_estudiante -> validar_ppa_minimo -> inscribir(SIS206)

codigo=1001 -> EXITO: Juan Mamani inscrito en SIS206
codigo=9999 -> FALLO: Código 9999 no existe
codigo=1002 -> FALLO: PPA 10.5 < 12.0 (mínimo para inscribirse)
codigo=1003 -> FALLO: Luis Condori ya está inscrito en SIS206

map() sobre un resultado exitoso:
  nombre en mayúsculas = JUAN MAMANI
map() sobre un fallo (la función NO se ejecuta):
  llamadas a la función = 0, error propagado = Código 9999 no existe
```

▶ SALIDA — PYTHON

```
Pipeline: buscar_estudiante -> validar_ppa_minimo -> inscribir(SIS206)

codigo=1001 -> EXITO: Juan Mamani inscrito en SIS206
codigo=9999 -> FALLO: Código 9999 no existe
```

```
codigo=1002 -> FALLO: PPA 10.5 < 12.0 (mínimo para inscribirse)
codigo=1003 -> FALLO: Luis Condori ya está inscrito en SIS206

map() sobre un resultado exitoso:
  nombre en mayúsculas = JUAN MAMANI
map() sobre un fallo (la función NO se ejecuta):
  llamadas a la función = 0, error propagado = Código 9999 no existe

Result es inmutable: FrozenInstanceError
```

Preguntas de análisis

Pregunta	Respuesta
¿Por qué `.map()` no ejecuta la siguiente función si el Result es un fallo?	Porque antes de llamar a la función comprueba si hay éxito. Si es un fallo, devuelve **otro fallo con el mismo mensaje** sin invocarla (cortocircuito). La salida lo prueba: con el código 9999 el contador de llamadas es `0`. En el pipeline, el 1002 no llega a `inscribir` porque falla en `validar_ppa_minimo`.
¿En qué se diferencia `Result.fallo()` de lanzar una excepción en términos de flujo de control?	`Result.fallo()` es un **retorno normal**: el control vuelve al llamador, que debe decidir qué hacer; el fallo es visible en el tipo de retorno. Una excepción es un **salto no local**: se desenrolla la pila hasta el `catch` más cercano, saltando todo el código intermedio, y puede ignorarse sin que la firma lo advierta.
¿Cuándo elegiría `Result(T)` sobre `throw/raise` en su Proyecto Integrador?	Para los **fallos esperados** que ocurren en el uso normal: buscar un estudiante por código (`RepositorioEstudiantes.buscar`), validar el PPA mínimo, comprobar datos de un formulario. Dejaría `throw/raise` para precondiciones violadas en constructores y mutadores, y para fallos de entorno (archivos, red).

Análisis. El pipeline maneja correctamente el éxito (1001) y tres fallos distintos: código inexistente (9999), PPA insuficiente (1002) y ya inscrito (1003). El error se revisa **una sola vez, al final**, sin `try/catch` en cada paso. `map` transforma un valor (`nombre → NOMBRE`) y `andThen` encadena pasos que ya devuelven `Result`; separar ambos evita el `Result⟨Result⟨T⟩⟩` del código original.

7. Actividad 4 — Verificación de contratos con pruebas unitarias

Objetivo. Comprobar con pruebas que los contratos de `Matricula` se cumplen: casos válidos que deben funcionar y casos inválidos que deben lanzar la excepción correcta. Se escribieron **10 pruebas en C++** (`assert` en un `main` de pruebas) y **11 en Python** (`pytest`).

Prueba	Qué verifica del contrato
constructor válido	Un objeto correcto queda activo y con los cursos dados
sin cursos / 7 cursos	Precondición de capacidad → `ErrorCapacidad`
cursos duplicados / fecha futura	Precondiciones de validación → `ErrorValidacion`
agregar curso exitoso	Poscondición: el curso queda y el tamaño crece en 1
agregar curso duplicado	Precondición → `ErrorValidacion`
eliminar último curso	Invariante: no puede quedar sin cursos → `ErrorValidacion`
anular y modificar	Matrícula anulada no se modifica → `ErrorAcademico`
diagnóstico	Texto `ACTIVA` / `ANULADA` (C++) · invariante rota detectada (Python)

C++ — cpp/actividad4/tests_matricula.cpp

```
void test_agregar_curso_duplicado() {
    Matricula m("20210500", {"SIS206"});
    try {
        m.agregarCurso("SIS206");  // duplicado — debe lanzar
        assert(false && "No lanzó excepción");
    } catch (const ErrorValidacion& e) {
        PASS(std::string("duplicado: ") + e.what());
    }
}
```

PYTHON — python/tests/test_matricula.py

```
def test_agregar_curso_duplicado():
    m = Matricula("20210500", ["SIS206"], HOY)
    with pytest.raises(ErrorValidacion):
        m.agregar_curso("SIS206")
```

▶ **SALIDA — PRUEBAS C++**

```
[PASS] constructor válido
[PASS] constructor sin cursos: [Académico] matricula: capacidad agotada (0/6)
[PASS] constructor con 7 cursos: [Académico] matricula: capacidad agotada (7/6)
[PASS] constructor con duplicados: [Académico] [Validación] Cursos duplicados en la matrícula
[PASS] fecha futura: [Académico] [Validación] Fecha de matrícula futura no permitida
[PASS] agregar curso exitoso
[PASS] duplicado: [Académico] [Validación] Curso 'SIS206' ya está en la matrícula
[PASS] eliminar último curso: [Académico] [Validación] La matrícula no puede quedar sin cursos
(mínimo 1)
[PASS] anulada: [Académico] Matrícula anulada — no se puede modificar
[PASS] diagnóstico ACTIVA/ANULADA

Todas las pruebas pasaron (10).
```

▶ **SALIDA — PRUEBAS PYTHON**

```
collected: tests/test_matricula

tests/test_matricula.py::test_agregar_curso_duplicado PASSED
tests/test_matricula.py::test_agregar_curso_exitoso PASSED
```

```
tests/test_matricula.py::test_anular_y_modificar PASSED
tests/test_matricula.py::test_constructor_demasiados_cursos PASSED
tests/test_matricula.py::test_constructor_duplicados PASSED
tests/test_matricula.py::test_constructor_fecha_futura PASSED
tests/test_matricula.py::test_constructor_sin_cursos PASSED
tests/test_matricula.py::test_constructor_valido PASSED
tests/test_matricula.py::test_diagnostico_detecta_invariante_rota PASSED
tests/test_matricula.py::test_eliminar_ultimo_curso_no_permitido PASSED
tests/test_matricula.py::test_invariante_directo PASSED

======== 11 passed, 0 failed in 0.00s ========
```

Las pruebas Python están escritas para `pytest` (`pytest tests -v`); este resultado se obtuvo con un ejecutor mínimo compatible que llama a cada función `test_*`.

> **Análisis.** Las 10 pruebas C++ y las 11 de Python pasan. Cada caso inválido comprueba el **tipo exacto** de la excepción (no solo que falle), de modo que si el contrato cambia, la prueba lo delata. Es importante compilar las pruebas C++ **sin** `-DNDEBUG`: con él, `assert` desaparece y las pruebas no comprobarían nada.

8. Actividad 5 — Proyecto Integrador: robustez en todas las capas

Objetivo. Aplicar al Proyecto Integrador una jerarquía de errores propia, precondiciones con `throw`, invariantes, búsquedas con `Result(T)` y pruebas de contratos, en C++ y Python.

> **Nota sobre el proyecto.** La guía no incluye el código del Proyecto Integrador, por lo que se modeló un pequeño **sistema de gestión académica** (Estudiante, Seccion, RepositorioEstudiantes) que reúne todo lo pedido. Sus clases y errores se pueden trasladar directamente a las del proyecto real.

Requisito de la guía	Cómo se cumple
C++: jerarquía con ≥ 4 clases	`errores_dominio.hpp`: `ErrorDominio` + 4 hijas (`ErrorDatoInvalido`, `ErrorEntidadDuplicada`, `ErrorEntidadNoEncontrada`, `ErrorCupoAgotado`)
Precondiciones con `throw` en constructores y mutadores	`Estudiante`, `Seccion` y `RepositorioEstudiantes` validan antes de modificar
Python: `excepciones.py` y `_verificar_invariante()` en ≥ 2 clases	`proyecto_integrador/excepciones.py`; invariante en `Estudiante` y `Seccion`
Refactorizar una búsqueda a `Result(T)`	`RepositorioEstudiantes.buscar()` en ambos lenguajes (se conserva `obtener()` con excepción para comparar)
≥ 6 pruebas (3 válidas y 3 que lanzan)	8 pruebas en C++ (`tests_dominio.cpp`) y 8 en Python (`test_dominio.py`)

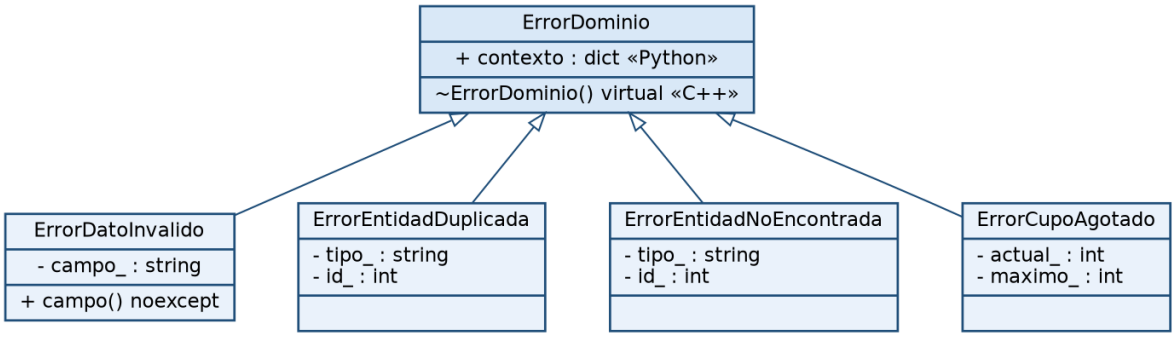


Figura 5. Jerarquía de errores del Proyecto Integrador (ErrorDominio y sus cuatro hijas).

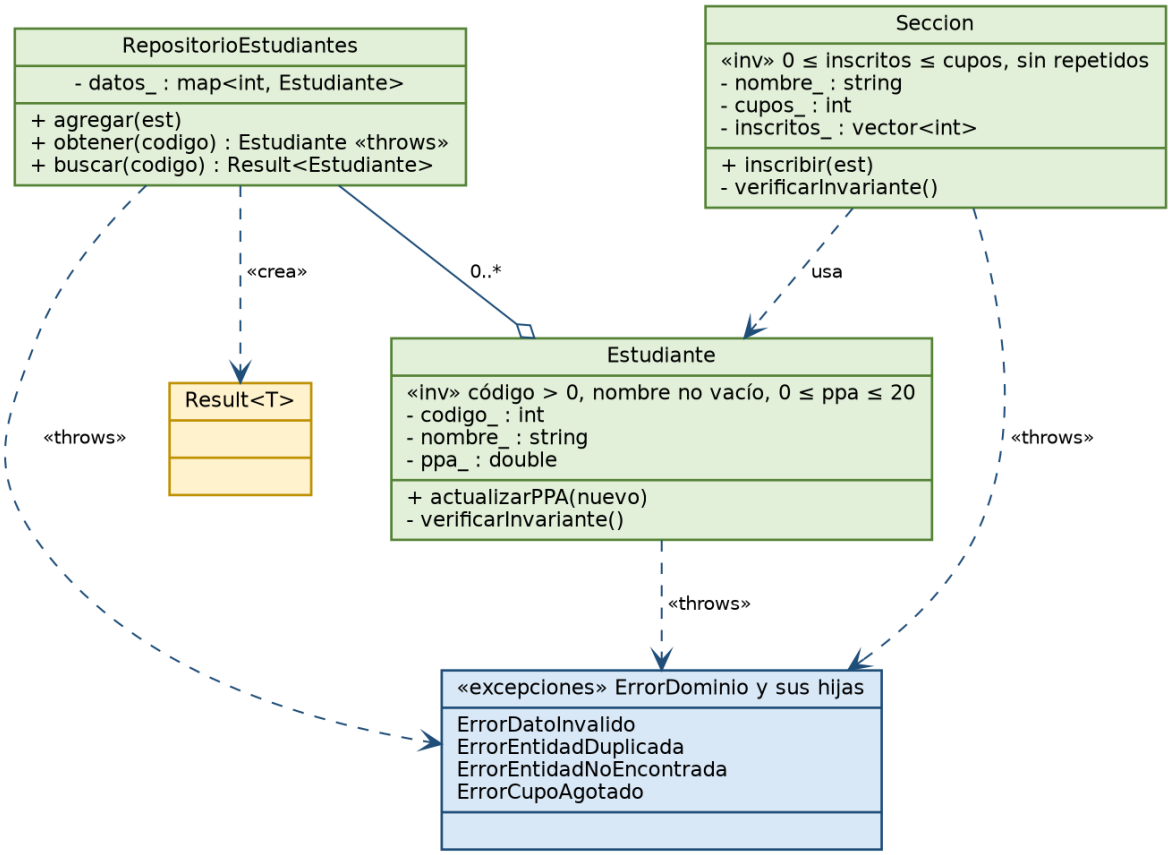


Figura 6. Clases del Proyecto Integrador: Estudiante, Seccion, RepositorioEstudiantes y Result.

C++ — cpp/actividad5/modelos.hpp

```
Result<Estudiante> buscar(int codigo) const {
    auto it = datos_.find(codigo);
    if (it == datos_.end())
        return Result<Estudiante>::fallo("Estudiante id=" + std::to_string(codigo) + " no
encontrado");
    return Result<Estudiante>::ok(it->second);
}
```

PYTHON — python/proyecto_integrador/modelos.py

```
def buscar(self, codigo: int) -> Result[Estudiante]:
    """Versión refactorizada: 'no encontrado' es un fallo ESPERADO -> Result."""
    est = self._datos.get(codigo)
    return Result.ok(est) if est else Result.fallo(f"Estudiante id={codigo} no encontrado")
```

▶ **SALIDA — C++**

```
Repositorio con 2 estudiantes

--- Result<T> en la búsqueda (fallo esperado) ---
buscar(1001) -> Juan Mamani
buscar(9999) -> FALLO: Estudiante id=9999 no encontrado

--- Excepciones en las precondiciones (error de uso) ---
ErrorDatoInvalido(campo=ppa): [Dominio] Dato inválido en 'ppa': debe estar en [0, 20]
ErrorEntidadDuplicada: [Dominio] Estudiante id=1001 ya existe
ErrorEntidadNoEncontrada: [Dominio] Estudiante id=4242 no encontrado

--- Seccion con DbC ---
Inscritos 2, cupos libres 0
ErrorCupoAgotado: [Dominio] Sección POO II - A sin cupos (2/2)
```

```
Captura por familia (ErrorDominio&): [Dominio] Sección X sin cupos (2/2)
```

▶️ SALIDA — PYTHON

```
Repositorio con 2 estudiantes

--- Result[T] en la búsqueda (fallo esperado) ---
buscar(1001) -> Juan Mamani
buscar(9999) -> FALLO: Estudiante id=9999 no encontrado

--- Excepciones en las precondiciones (error de uso) ---
ErrorDatoInvalido(campo=ppa): Dato inválido en 'ppa': debe estar en [0, 20]  <- contexto:
{campo='ppa'}
ErrorEntidadDuplicada: Estudiante id=1001 ya existe  <- contexto: {tipo='Estudiante', id=1001}
ErrorEntidadNoEncontrada: Estudiante id=4242 no encontrado  <- contexto: {tipo='Estudiante',
id=4242}

--- Seccion con DbC ---
Inscritos 2, cupos libres 0
ErrorCupoAgotado: Sección POO II - A sin cupos (2/2)  <- contexto: {actual=2, maximo=2,
seccion='POO II - A'}
Captura por familia (ErrorDominio): Sección X sin cupos (2/2)  <- contexto: {actual=2,
maximo=2, seccion='X'}
```

▶️ SALIDA — PRUEBAS C++

```
[PASS] estudiante válido y actualizarPPA
[PASS] sección inscribe dentro del cupo
[PASS] buscar() devuelve Result ok / fallo sin lanzar
[PASS] PPA inválido: [Dominio] Dato inválido en 'ppa': debe estar en [0, 20]
[PASS] cupo agotado: [Dominio] Sección S sin cupos (1/1)
[PASS] duplicado: [Dominio] Estudiante id=1 ya existe
[PASS] obtener inexistente (por familia): [Dominio] Estudiante id=77 no encontrado
[PASS] inscripción repetida: [Dominio] Inscripción id=1 ya existe

Todas las pruebas pasaron (8).
```

▶️ SALIDA — PRUEBAS PYTHON

```
collected: tests/test_dominio

tests/test_dominio.py::test_buscar_result PASSED
tests/test_dominio.py::test_cupo_agotado PASSED
tests/test_dominio.py::test_estudiante_valido PASSED
tests/test_dominio.py::test_inscripcion_repetida PASSED
tests/test_dominio.py::test_obtener_no_encontrado_por_familia PASSED
tests/test_dominio.py::test_ppa_fuera_de_rango PASSED
tests/test_dominio.py::test_repositorio_duplicado PASSED
tests/test_dominio.py::test_seccion_inscribe PASSED

======== 8 passed, 0 failed in 0.03s ========
```

Análisis. Se ve el reparto de responsabilidades: `buscar()` devuelve `Result` porque "no encontrado" es un fallo **esperado**, mientras que crear un `Estudiante` con PPA 25, repetir un código o inscribir en una sección llena lanza una excepción de la jerarquía `ErrorDominio`. Un solo `catch (const ErrorDominio&)` atrapa a todas las hijas ("captura por familia"). Las 8 pruebas de cada lenguaje pasan.

9. Preguntas de reflexión

1. ¿Por qué el destructor de la clase base de excepciones debe ser virtual?

Si se destruye un `ErrorCalificacion` a través de un puntero `ErrorAcademico*` y el destructor de la base **no es virtual**, el `delete` tiene **comportamiento indefinido**. En la práctica, en tiempo de ejecución solo se ejecuta el destructor de la base: los miembros de la clase derivada (por ejemplo, el `std::string tipo_` de `ErrorRecursoNoEncontrado`) nunca se destruyen, lo que provoca fugas de memoria, y el `operator delete` puede recibir un tamaño equivocado. Con destructor virtual, la llamada se resuelve dinámicamente al destructor del tipo real y luego encadena hacia las bases. En este proyecto `std::exception` ya tiene destructor virtual, por lo que `ErrorAcademico` lo hereda; se declara explícito (`~ErrorAcademico() override = default`) para dejar constancia del requisito.

2. ¿Por qué assert para poscondiciones e invariantes, y throw para precondiciones?

La diferencia es **de quién es la culpa y si puede ocurrir en producción**. Una precondición depende de lo que envía quien llama (datos del usuario, archivos, otros módulos): puede fallar aunque nuestro código sea correcto, así que debe comprobarse **siempre** y de forma recuperable (`throw`). Una poscondición o una invariante verifican lo que hace **nuestro propio código**: si fallan, existe un bug interno, que se debe detectar en desarrollo y pruebas, no "manejar" en producción. Por eso son `assert`, que con `-DNDEBUG` desaparece y no cuesta rendimiento. El costo del trato asimétrico es que en modo release esas verificaciones no están activas, por lo que las pruebas deben ejecutarse con los `assert` habilitados.

3. ¿Qué estrategia produce sistemas más seguros: Result o excepciones?

Depende del contexto; no hay respuesta universal. `Result` obliga a tratar el fallo en el punto de uso y lo hace visible en la firma, lo cual es ideal para fallos esperados y locales. Pero un `Result` también se puede ignorar sin inspeccionarlo (por eso se marcó `[[nodiscard]]` en C++). Las excepciones pueden ignorarse en el punto de llamada, pero si nadie las atrapa el programa termina de forma ruidosa, de modo que el error **no se pierde en silencio**, y atraviesan varias capas sin código intermedio. La combinación más segura es la usada aquí: `Result` para los fallos de negocio esperados y excepciones para errores de uso, fallos de entorno y violaciones de contrato.

10. Cumplimiento de la rúbrica

Criterio	Evidencia en el proyecto	Peso
Jerarquía C++ (Act. 1A)	Compila con `-Wall -Wextra` sin avisos · 7 clases · destructor virtual · captura selectiva en `actividad1/main.cpp`	20%
Jerarquía Python (Act. 1B)	Contexto en `dict` · `raise ... from ...` · captura por familia · `__str__` útil	15%
DbC Matricula C++ (Act. 2)	Pre / pos / invariante · `verificarInvariante()` · `noexcept` en getters	20%
DbC Matricula Python (Act. 2)	`_verificar_invariante()` · `assert` de poscondición · `diagnostico()`	10%
Patrón Result (Act. 3)	Result completo con `map` y `andThen` · pipeline de 3 pasos · preguntas respondidas	15%
Pruebas unitarias (Act. 4)	10 pruebas C++ + 11 pytest · casos válidos e inválidos	10%
Proyecto Integrador (Act. 5)	Jerarquía + DbC + Result en C++ y Python · 8 + 8 pruebas	10%

11. Conclusiones

- Una **jerarquía de excepciones** bien diseñada permite capturar por familia o por caso concreto, y transporta contexto útil (valor, código, límite) para diagnosticar.
- El **Diseño por Contrato** separa responsabilidades: las precondiciones culpan a quien llama; las poscondiciones y la invariante protegen la corrección interna de la clase.
- El **patrón Result** hace explícito el fallo esperado y permite encadenar operaciones comprobando el error una sola vez, al final.
- El **diagnóstico del marco teórico** mostró que el código de una guía también puede tener errores (aquí, tres de compilación): compilar, ejecutar y probar siempre es parte del trabajo.

12. Referencias

cppreference.com. (2026). std::exception. https://en.cppreference.com/w/cpp/error/exception

cppreference.com. (2026). std::variant. https://en.cppreference.com/w/cpp/utility/variant

Meyer, B. (1997). Object-Oriented Software Construction (2nd ed.). Prentice Hall.

Stroustrup, B. (2024). Programming: Principles and Practice Using C++ (3rd ed.). Addison-Wesley.

Python Software Foundation. (2026). Built-in Exceptions. https://docs.python.org/3/library/exceptions.html

Martin, R. C. (2008). Clean Code. Prentice Hall.

Hatcliff, J. et al. (2012). Behavioral interface specification languages. ACM Computing Surveys, 44(3). https://doi.org/10.1145/2187671.2187678